

0. Contributions: Sriram Nair (all)**1. Design Decisions***a. How were states represented:*

Each state is a tuple of the robot's position and the dirty cells' frozenset. ((robot row, robot column), frozenset(dirty)). Frozensets were used because they are hashable, while, "a plain set is not hashable and will raise TypeError the first time you try to add it to your explored set. (From the implementation note)"

*b. Reopening in A**

A* uses priority queue with $f = g + h$, and dictionary g for finding the cheapest cost found at that point. The dictionary came_from is used for rebuilding the path at the end to return with, and was mentioned in the comments of the implementation of the zip file provided. If a state with a lower g is reached, update g and the came_from dictionary, then push it again. After that, check for the goal when a state is .pop()'ed to make sure A* returns an optimal path

c. Differences from the Spec (assuming spec means the pdf? Not 100% sure)

IDA* stops if the threshold goes above (number of dirty cells)*(number of free cells), so it doesn't loop infinitely. Also added a quicker exit if a dirty cell was on a wall cause that would have no solution (looked for other ways to get faster no sol.s but didn't think of any)

d. Expanded vs Generated

I counted nodes expanded when popped from the frontier, had successors generated, and isn't the goal, and not nodes generated for DFS, A*, and IDA*.

2. Heuristic Analysis*a. h1 admissibility*

h1 returns the number of dirty cells left (len(state[1])), which is the lower bound for the number of cleaning actions needed. Each clean costs 1 and cleans the cell the Roomba is on, so the Roomba must spend (number of dirty cells left) on cleaning. The cost is increased by moves, so the remaining cost cannot be less than h1

b. h2 admissibility

h2 returns (dirty cells left)+Manhattan distance to nearest dirty cell, and returns 0 when no dirty cells. The dirty cells part is the lower bound for cleaning cost, like h1, and the distance part is the lower bound on movement. The Roomba has to reach some dirty cell before it can clean and the moves are changing the row and column by 1 each time. It cannot both clean and move (unlike a Roomba) so adding them together doesn't have any overlap and the remaining cost cannot be less than h2.

c. Dominance:

h2 dominates h1 because $h2 = h1 + a$ (always positive) distance, so h2 is always $\geq h1$ in all states. Dominance says that A* with h2 expands the $\leq$ the number of nodes with h1, as A* only expands nodes where $g + h$ is below the current optimal cost, and the larger the h, the more nodes skipped.

d. I didn't do h3, so nonapplicable

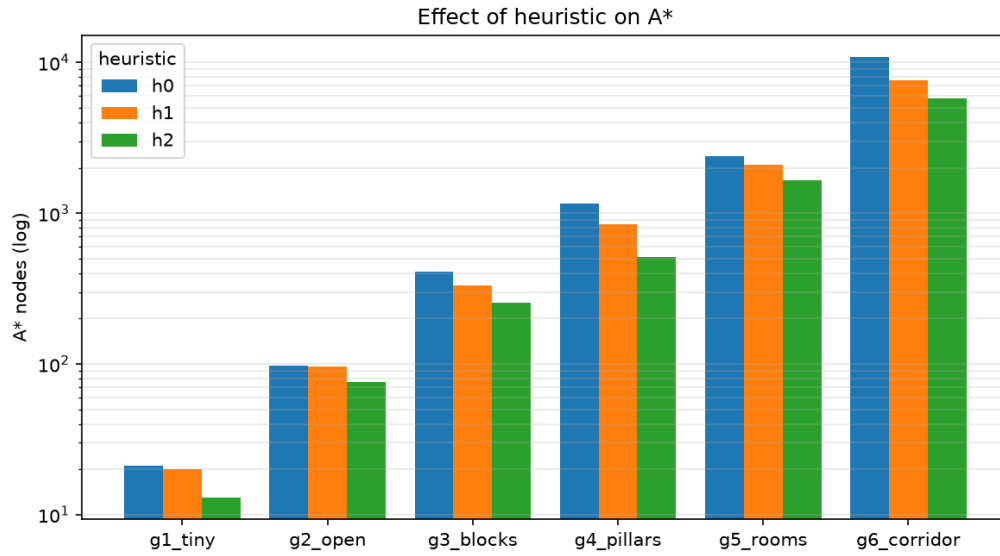
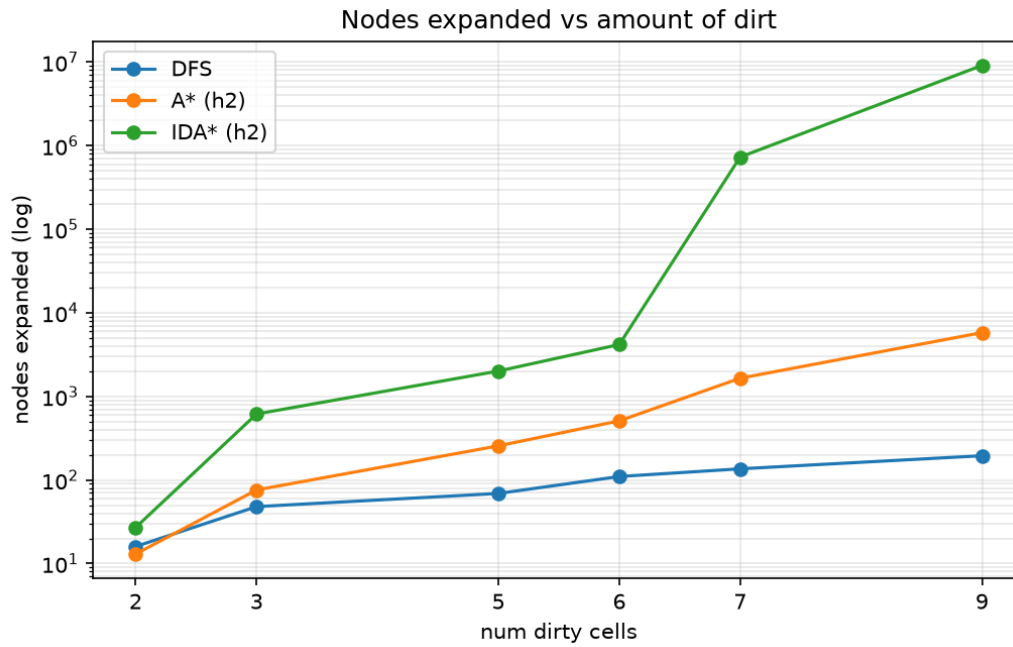
CS3810 Mini-Project 1

Sriram Nair

3. Experimental Results:

grid	rows	cols	n_dirty	algorithm	heu	status	cost	nodes_exp	max_f	iterations	seconds
g1_tiny	3	3	2	dfs	-	ok	12	16	5		0
g1_tiny	3	3	2	astar	h0	ok	7	21	4		0.0001
g1_tiny	3	3	2	astar	h1	ok	7	20	5		0.0001
g1_tiny	3	3	2	astar	h2	ok	7	13	4		0.0001
g1_tiny	3	3	2	astar	h3	SKIP					
g1_tiny	3	3	2	idastar	h0	ok	7	95		8	0.0002
g1_tiny	3	3	2	idastar	h1	ok	7	52		6	0.0001
g1_tiny	3	3	2	idastar	h2	ok	7	27		4	0.0001
g1_tiny	3	3	2	idastar	h3	SKIP					
g2_open	4	4	3	dfs	-	ok	36	48	32		0.0003
g2_open	4	4	3	astar	h0	ok	12	98	17		0.0003
g2_open	4	4	3	astar	h1	ok	12	96	18		0.0003
g2_open	4	4	3	astar	h2	ok	12	76	21		0.0003
g2_open	4	4	3	astar	h3	SKIP					
g2_open	4	4	3	idastar	h0	ok	12	15586		13	0.0399
g2_open	4	4	3	idastar	h1	ok	12	3821		10	0.0087
g2_open	4	4	3	idastar	h2	ok	12	616		4	0.002
g2_open	4	4	3	idastar	h3	SKIP					
g3_blocks	4	4	5	dfs	-	ok	43	69	34		0.0002
g3_blocks	4	4	5	astar	h0	ok	15	411	61		0.0016
g3_blocks	4	4	5	astar	h1	ok	15	331	71		0.0024
g3_blocks	4	4	5	astar	h2	ok	15	256	87		0.0015
g3_blocks	4	4	5	astar	h3	SKIP					
g3_blocks	4	4	5	idastar	h0	ok	15	29281		16	0.066
g3_blocks	4	4	5	idastar	h1	ok	15	4463		11	0.0103
g3_blocks	4	4	5	idastar	h2	ok	15	2010		8	0.0068
g3_blocks	4	4	5	idastar	h3	SKIP					
g4_pillars	5	5	6	dfs	-	ok	80	110	50		0.0003
g4_pillars	5	5	6	astar	h0	ok	19	1160	161		0.0035
g4_pillars	5	5	6	astar	h1	ok	19	850	201		0.0038
g4_pillars	5	5	6	astar	h2	ok	19	510	191		0.0026
g4_pillars	5	5	6	astar	h3	SKIP					
g4_pillars	5	5	6	idastar	h0	ok	19	58097		20	0.1181
g4_pillars	5	5	6	idastar	h1	ok	19	8883		14	0.0168
g4_pillars	5	5	6	idastar	h2	ok	19	4174		12	0.0183
g4_pillars	5	5	6	idastar	h3	SKIP					
g5_rooms	5	5	7	dfs	-	ok	92	136	68		0.0004
g5_rooms	5	5	7	astar	h0	ok	25	2403	237		0.0074
g5_rooms	5	5	7	astar	h1	ok	25	2097	274		0.0077
g5_rooms	5	5	7	astar	h2	ok	25	1651	380		0.0068
g5_rooms	5	5	7	astar	h3	SKIP					
g5_rooms	5	5	7	idastar	h0	ok	25	17817953		26	40.794
g5_rooms	5	5	7	idastar	h1	ok	25	1802996		19	4.217
g5_rooms	5	5	7	idastar	h2	ok	25	731040		17	2.8911
g5_rooms	5	5	7	idastar	h3	SKIP					
g6_corridor	6	6	9	dfs	-	ok	121	195	90		0.0006
g6_corridor	6	6	9	astar	h0	ok	32	10873	940		0.0337
g6_corridor	6	6	9	astar	h1	ok	32	7665	1066		0.022
g6_corridor	6	6	9	astar	h2	ok	32	5797	1282		0.0283
g6_corridor	6	6	9	astar	h3	SKIP					
g6_corridor	6	6	9	idastar	h0	TIMEOUT					60.0179
g6_corridor	6	6	9	idastar	h1	TIMEOUT					60.0186
g6_corridor	6	6	9	idastar	h2	ok	32	9126492		21	45.6173
g6_corridor	6	6	9	idastar	h3	SKIP					

CS3810 Mini-Project 1
Sriram Nair



4. Discussion Questions

- a. *Why DFS expands fewer nodes than A* on some grids, and why its solutions are much worse. Explain both sides*

DFS stops at the first goal it can reach and doesn't compare other plans with any others. Its solutions are much worse since A* always get the optimal while DFS could be doing a very roundabout path and reach the goal. Evidence is plan costs are 12:7 on g1, 80:19 on g4, and 121:32 on g6, which is significantly higher plan costs than the optimal A*.

- b. *How A* node count changes from h0 to h1 to h2, whether that matches the dominance prediction, and if not, why?*

A* expanded fewer nodes as the heuristic changed between h0 to h1 to h2. That's what dominance predicts because a heuristic that's at least as large as the previous one lets A* skip more nodes, meaning better dominance. The h0 to h1 jump was small since h1 just counts dirty cells, not where they are. h2 helped a lot more since it gave the distances to the dirty cells to make the Roomba go towards them

- c. *Why anyone would use IDA* despite it expanding more nodes, with a concrete situation where it's the right choice*

A* stores the entire frontier and its closed set, making it have a larger memory requirement. If you're working on a nanobot where memory is much more important than time/optimal route, would rather use IDA* as regular A*'s tables wouldn't fit but IDA*'s stack would

- d. *Formula for reachable states in terms of grid size and number of dirty cells, and how that explains the blow-up.*

A state is the Roomba position and the dirty cells set, so the count is (free cells)*2^(dirty cells). The count is linear in the grid but exponential with the dirt

- e. Which grid was hardest and what structural property made it hard

G6 was the hardest because:

- i. It's the largest grid, so the most amount of cells needed to be looked at
- ii. It has the most dirt, 9 dirt cells
- iii. The structure has it force the path all the way to the right because of the single gap, so the robot has to go through the gap but finding the gap adds overhead.

- f. **Roomba question:** *pick one assumption (perfect map, known dirt, deterministic actions, unlimited battery), explain what breaks in your implementation without it, and what class of technique would be needed instead (one paragraph)*

Unlimited battery would not allow some optimal plans as it would need to recharge to go back. A technique to be implemented would be recharging, where it remembers where to recharge and path back to that, and knowing how much energy is needed to path back to the recharge point so to never go below that minimum amount. It may also need to have multiple different paths to clean different spots

5. Reflection

a. What broke:

I didn't have the latest version of matplotlib or pandas installed on my working computer, and was running into trouble implementing that. I had to uninstall and reinstall both. I didn't realize that actions had to be reversed in `_reconstruct` method in `search.py`.

b. How you found it

For the first one, the graph would fail to be made and would throw an error when running:
`python experiments.py --analyze`

For the second, it would show up in the 2b test that it failed, and I kept looking through the helper methods to see if there was a problem I was missing and I added a simple `print()` message after each `reconstruct` to see the path and saw it was backwards, and proceeded to fix it.

c. What you would do differently

Honestly nothing comes to mind, the project went well, only real thing was not being well versed in matplotlib and pandas.